
Entrenamiento y visualización de una red neuronal en R

Como se vio en los temas anteriores, conceptos de redes neuronales e inteligencia artificial, y procesos de aprendizaje en redes neuronales, el entrenamiento de un modelo de red neuronal constituye la base para construir una red neuronal.

La retropropagación y retroalimentación son las técnicas que se utilizan para determinar los pesos y sesgos del modelo. Los pesos nunca pueden ser cero, pero los sesgos pueden ser cero. Para empezar, los pesos se inicializan en un número aleatorio y, por descenso de gradiente, se minimizan los errores; obtenemos un conjunto de los mejores pesos y sesgos posibles para el modelo.

Una vez que se entrena el modelo usando cualquiera de las funciones R, podemos pasar las variables independientes para predecir la variable objetivo o desconocida. En este documento, utilizaremos un conjunto de datos disponible públicamente para entrenar, probar y visualizar un modelo de red neuronal. Se cubrirán los siguientes elementos:

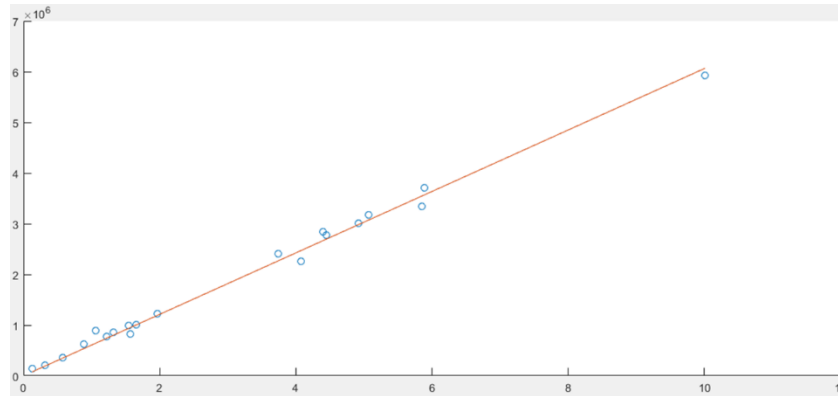
- Entrenamiento, prueba y evaluación de un conjunto de datos usando el modelo NN (Neural Network)
- Visualizando el modelo NN
- Parada anticipada
- Evitando el sobreajuste
- Generalización de la NN
- Escalado de parámetros de la NN
- Modelos de conjunto

Al final del documento, entenderemos cómo entrenar, probar y evaluar un conjunto de datos utilizando el modelo NN. Aprenderemos a visualizar el modelo NN en un entorno R. Cubriremos los conceptos como parada anticipada, evitar el sobreajuste, generalización de NN y escalado de parámetros de NN.

Ajuste de datos con red neuronal

El ajuste de datos es el proceso de construir una curva o una función matemática que tiene la mejor coincidencia con un conjunto de puntos recopilados previamente. El ajuste de la curva puede relacionarse tanto con interpolaciones, donde se requieren puntos de datos exactos, como con suavizado, donde se construye una función plana que se aproxima a los datos. Las curvas aproximadas obtenidas del ajuste de datos se pueden utilizar para ayudar a mostrar

datos, para predecir los valores de una función donde no hay datos disponibles y para resumir la relación entre dos o más variables. En la siguiente figura se muestra una interpolación lineal de los datos recopilados:



El ajuste de datos es el proceso de entrenar una red neuronal en un conjunto de entradas para producir un conjunto asociado de salidas objetivo. Una vez que la red neuronal ha ajustado los datos, forma una generalización de la relación entrada-salida y se puede utilizar para generar salidas para entradas en las que no se entrenó.

El consumo de combustible de los vehículos siempre ha sido estudiado por los principales fabricantes de todo el planeta. En una era caracterizada por problemas de repostaje de petróleo e incluso mayores problemas de contaminación del aire, el consumo de combustible de los vehículos se ha convertido en un factor clave. En este ejemplo, construiremos una red neuronal con la finalidad de predecir el consumo de combustible de los vehículos según determinadas características.

Para hacer esto, usa el conjunto de datos Auto contenido en el paquete ISLR. El conjunto de datos Auto contiene el consumo de combustible, los caballos de fuerza y otra información para 392 vehículos. Es un marco de datos con 392 observaciones sobre las siguientes nueve variables:

- **mpg:** Millas por galón
- **cylinders:** Número de cilindros entre 4 y 8
- **displacement:** desplazamiento del motor (pulgadas cúbicas)
- **horsepower:** caballos de fuerza del motor
- **weight:** peso del vehículo (libras)
- **acceleration:** tiempo para acelerar de 0 a 60 mph (seg)
- **year:** año del modelo (módulo 100)
- **origin:** origen del coche (American, European, Japanese)
- **name:** Nombre del vehículo

El siguiente es el código que usaremos en este ejemplo:

```
> install.packages("ISLR")
> install.packages("neuralnet")

> library("neuralnet")
> library("ISLR")

> data = Auto
> View(data)

> plot(data$weight, data$mpg, pch=data$origin,cex=2)
> par(mfrow=c(2,2))
> plot(data$cylinders, data$mpg, pch=data$origin,cex=1)
> plot(data$displacement, data$mpg, pch=data$origin,cex=1)
> plot(data$horsepower, data$mpg, pch=data$origin,cex=1)
> plot(data$acceleration, data$mpg, pch=data$origin,cex=1)

> mean_data <- apply(data[1:6], 2, mean)
> sd_data <- apply(data[1:6], 2, sd)

> data_scaled <- as.data.frame(scale(data[,1:6],center = mean_data, scale = sd_data))
> head(data_scaled, n=20)

> index = sample(1:nrow(data),round(0.70*nrow(data)))
> train_data <- as.data.frame(data_scaled[index,])
> test_data <- as.data.frame(data_scaled[-index,])

> n = names(data_scaled)
> f = as.formula(paste("mpg ~", paste(n[!n %in% "mpg"], collapse = " + ")))

> net = neuralnet(f,data=train_data,hidden=3,linear.output=TRUE)
> plot(net)

> predict_net_test <- compute(net,test_data[,2:6])
> MSE.net <- sum((test_data$mpg - predict_net_test$net.result)^2)/nrow(test_data)

> Lm_Mod <- lm(mpg~., data=train_data)
> summary(Lm_Mod)

> predict_lm <- predict(Lm_Mod,test_data)
> MSE.lm <- sum((predict_lm - test_data$mpg)^2)/nrow(test_data)
> par(mfrow=c(1,2))
```

```
> plot(test_data$mpg,predict_net_test$net.result,col='black',main='Real vs predicted for neural
network',pch=18,cex=4)
> abline(0,1,lwd=5)
```

Como es habitual, analizaremos el código línea por línea, explicando en detalle todas las características aplicadas para capturar los resultados.

Las dos primeras líneas del código inicial se utilizan para cargar las bibliotecas necesarias para ejecutar el análisis.

```
> library("neuralnet")
> library("ISLR")
```

La biblioteca de redes neuronales se utiliza para entrenar redes neuronales mediante retropropagación, **retropropagación resiliente (RPROP)** con o sin retroceso de peso, o la **versión globalmente convergente modificada (GRPROP)**. La función permite configuraciones flexibles mediante la elección personalizada de la función de error y activación. Además, se implementa el cálculo de pesos generalizados.

La biblioteca ISLR contiene un conjunto de data sets que se pueden utilizar libremente para nuestros ejemplos. Se trata de una serie de datos recopilados durante los principales estudios realizados por los centros de investigación.

```
> data = Auto
> View(data)
```

Este comando carga el conjunto de datos Auto, que, como anticipamos, está contenido en la biblioteca ISLR y lo guarda en un marco de datos determinado. Utiliza la función View para ver una visualización compacta de la estructura de un objeto R arbitrario. La siguiente captura de pantalla muestra algunos de los datos contenidos en el conjunto de datos Auto:

	row.names	mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	name
1	1	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
2	2	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
3	3	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
4	4	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
5	5	17.0	8	302.0	140	3449	10.5	70	1	ford torino
6	6	15.0	8	429.0	198	4341	10.0	70	1	ford galaxie 500
7	7	14.0	8	454.0	220	4354	9.0	70	1	chevrolet impala
8	8	14.0	8	440.0	215	4312	8.5	70	1	plymouth fury iii
9	9	14.0	8	455.0	225	4425	10.0	70	1	pontiac catalina
10	10	15.0	8	390.0	190	3850	8.5	70	1	amc ambassador dpl
11	11	15.0	8	383.0	170	3563	10.0	70	1	dodge challenger se
12	12	14.0	8	340.0	160	3609	8.0	70	1	plymouth 'cuda 340
13	13	15.0	8	400.0	150	3761	9.5	70	1	chevrolet monte carlo
14	14	14.0	8	455.0	225	3086	10.0	70	1	buick estate wagon (sw)
15	15	24.0	4	113.0	95	2372	15.0	70	3	toyota corona mark ii
16	16	22.0	6	198.0	95	2833	15.5	70	1	plymouth duster
17	17	18.0	6	199.0	97	2774	15.5	70	1	amc hornet
18	18	21.0	6	200.0	85	2587	16.0	70	1	ford maverick
19	19	27.0	4	97.0	88	2130	14.5	70	3	datsum pl510
20	20	26.0	4	97.0	46	1835	20.5	70	2	volkswagen 1131 deluxe sedan
21	21	25.0	4	110.0	87	2672	17.5	70	2	peugeot 504
22	22	24.0	4	107.0	90	2430	14.5	70	2	audi 100 ls
23	23	25.0	4	104.0	95	2375	17.5	70	2	saab 99e
24	24	26.0	4	121.0	113	2234	12.5	70	2	bmw 2002

Como puedes ver, la base de datos consta de 392 filas y 9 columnas. Las filas representan 392 vehículos comerciales de 1970 a 1982. Las columnas representan las 9 características recopiladas para cada automóvil, en orden: mpg, cylinders, displacement, horsepower, weight, acceleration, year, origin y name.

Análisis exploratorio

Antes de comenzar con el análisis de datos mediante la construcción y entrenamiento de una red neuronal, realizamos un análisis exploratorio para comprender cómo se distribuyen los datos y extraer conocimientos preliminares.

Podemos comenzar nuestro análisis exploratorio trazando una gráfica de predictores versus objetivo.

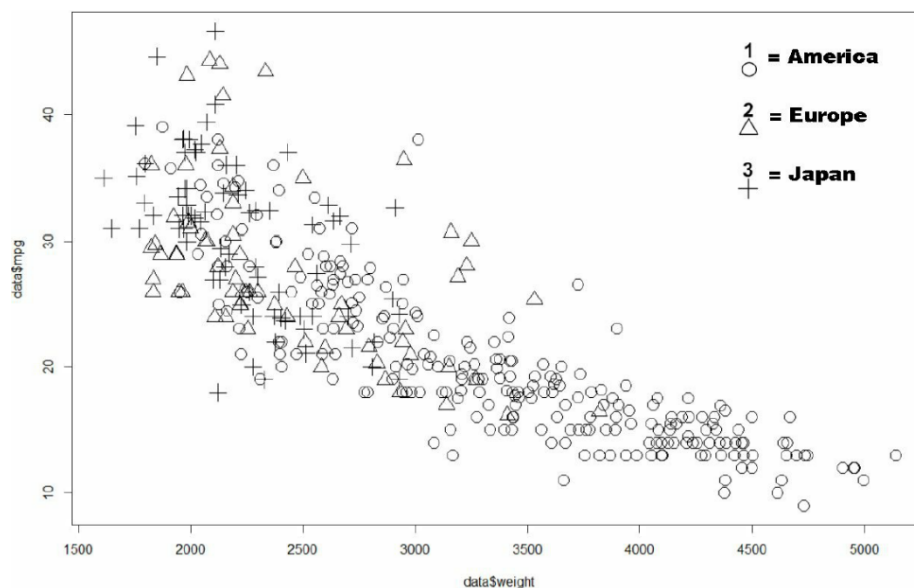
Recordamos al respecto que en nuestro análisis, los predictores son las siguientes variables: cylinders, displacement, horsepower, weight, acceleration, year, origin y name.

El objetivo es la variable mpg que contiene mediciones de las millas por galón de 392 autos de muestra.

Supongamos que queremos examinar el peso y el kilometraje de automóviles de tres orígenes diferentes, como se muestra en el siguiente gráfico, utilizando el siguiente código:

```
> plot(data$weight, data$mpg, pch=data$origin, cex=2)
```

Para trazar el gráfico, usamos la función plot(), especificando qué apuntar en el eje x (weight), qué apuntar en el eje y (mpg), y finalmente, en base a qué variable agrupar los datos (origin), como se muestra en la siguiente gráfica:



Recuerda que el número en la columna de origen corresponde a la siguiente zona: 1 = America, 2 = Europe y 3 = Japan). A partir del análisis del gráfico anterior, podemos encontrar que el consumo de combustible aumenta con el aumento de peso. Recordemos que el objetivo mide las millas por galón, entonces, cuántas millas recorren con un galón de combustible. De ello se deduce que cuanto mayor sea el valor de mpg (millas por galón), menor será el consumo de combustible.

Otra consideración que surge del análisis del plot es que los automóviles producidos en Estados Unidos son más pesados. De hecho, en la parte derecha del gráfico (que corresponde a valores de peso más altos), solo hay automóviles producidos en esa zona.

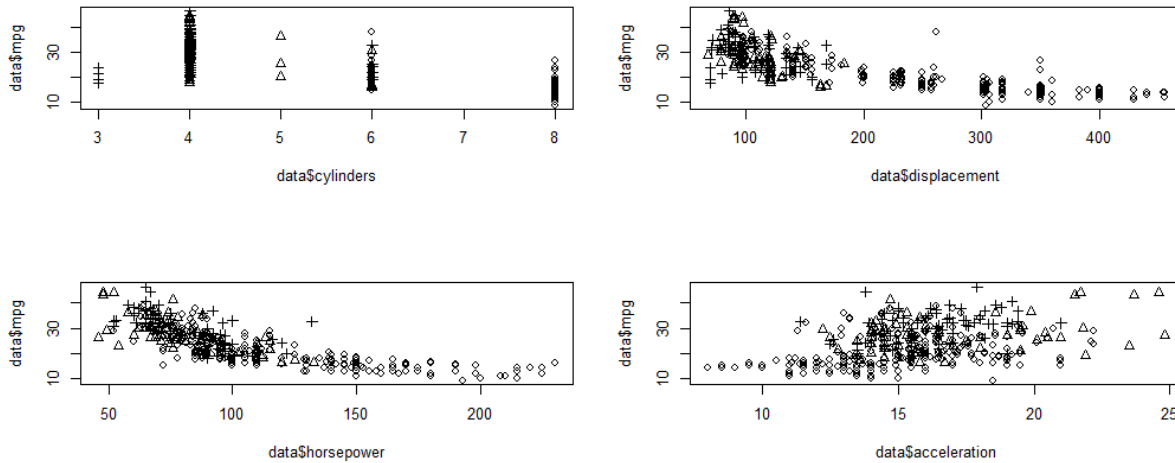
Por último, si centramos nuestro análisis en la parte izquierda del gráfico, en la parte superior que corresponde al menor consumo de combustible, encontramos en la mayoría de los casos coches japoneses y europeos. En conclusión, podemos señalar que los automóviles que tienen el menor consumo de combustible son los japoneses.

Ahora, veamos los otros gráficos, es decir, lo que obtenemos si graficamos los predictores numéricos restantes (cylinders, displacement, horsepower y acceleration) versus el objetivo (mpg).

```
> par(mfrow=c(2,2))
> plot(data$cylinders, data$mpg, pch=data$origin,cex=1)
> plot(data$displacement, data$mpg, pch=data$origin,cex=1)
> plot(data$horsepower, data$mpg, pch=data$origin,cex=1)
> plot(data$acceleration, data$mpg, pch=data$origin,cex=1)
```

Por razones de espacio, decidimos colocar los cuatro gráficos en uno. R facilita la combinación de varios gráficos en un gráfico general mediante la función `par()`. Usando la función `par()`, podemos incluir la opción `mfrow = c(nrows, ncols)` para crear una matriz de gráficos `nrows` x `ncols` que se llenan por filas. Por ejemplo, la opción `mfrow = c(3,2)` crea una gráfica de matriz con 3 filas y 2 columnas. Además, la opción `mfcow = c(nrows, ncols)` llena la matriz por columnas.

En la siguiente figura se muestran 4 gráficos dispuestos en una matriz de 2 filas y dos columnas:



Del análisis de la figura anterior, encontramos la confirmación de lo ya mencionado anteriormente. Podemos notar que los autos con mayor potencia tienen un mayor consumo de combustible. Lo mismo podemos decir sobre la cilindrada del motor; También en este caso, los vehículos de mayor cilindrada tienen un mayor consumo de combustible. Nuevamente, en Estados Unidos se producen automóviles con valores de potencia y desplazamiento más altos.

Por el contrario, los automóviles con valores de aceleración más altos tienen un menor consumo de combustible. Este hecho se debe al menor peso que tienen dichos coches. Por lo general, los autos pesados aceleran más lentamente.

Modelo de red neuronal

En el tema, Proceso de aprendizaje en redes neuronales, **escalamos los datos antes de construir la red**. En esa ocasión, señalamos que **es una buena práctica normalizar los datos antes de entrenar una red neuronal**. Con la normalización, se eliminan las unidades de datos, lo que te permite comparar fácilmente los datos de diferentes ubicaciones.

No siempre es necesario normalizar los datos numéricos. Sin embargo, se ha demostrado que cuando los valores numéricos se normalizan, la formación de redes neuronales suele ser más eficiente y conduce a una mejor predicción. De hecho, **si los datos numéricos no están normalizados y los tamaños de dos predictores están muy distantes, un cambio en el valor de un peso de red neuronal tiene mucha más influencia relativa en un valor más alto**.

Existen varias técnicas de estandarización; revisar Proceso de aprendizaje en redes neuronales, adoptamos la estandarización mínima-máxima. En este caso, adoptaremos la **normalización de puntajes Z**. Esta técnica consiste en restar la media de la columna a cada

valor de una columna y luego dividir el resultado por la desviación estándar de la columna. La fórmula para conseguirlo es la siguiente:

$$x_{escalada} = \frac{x - media}{desv. estandar}$$

En resumen, la puntuación Z (también llamada puntuación estándar) representa el número de desviaciones estándar con las que el valor de un punto de observación o datos es mayor que el valor medio de lo que se observa o mide. Los valores por encima de la media tienen puntuaciones Z positivas, mientras que los valores por debajo de la media tienen puntuaciones Z negativas. La puntuación Z es una cantidad sin dimensión, que se obtiene restando la media de la población de una única puntuación aproximada y luego dividiendo la diferencia por la desviación estándar de la población.

Antes de aplicar el método elegido para la normalización, debes calcular los valores de desviación estándar y media de cada columna de la base de datos. Para hacer esto, usamos la función `apply`. Esta función devuelve un vector o una matriz o una lista de valores obtenidos al aplicar una función a los márgenes de una matriz o matriz. Entendamos el significado de los argumentos utilizados.

```
> mean_data <- apply(data[1:6], 2, mean)
> sd_data <- apply(data[1:6], 2, sd)
```

La primera línea nos permite calcular la media de cada variable pasando a la segunda línea, lo que nos permite calcular la desviación estándar de cada variable. Veamos cómo usamos la función `apply()`. El primer argumento de la función de aplicación, especifica el conjunto de datos para aplicar la función, en nuestro caso, el conjunto de datos denominado `data`. En particular, solo hemos considerado las primeras seis variables numéricas; las demás las usaremos para otros fines. El segundo argumento debe contener un vector que proporcione los subíndices sobre los que se aplicará la función. En nuestro caso, uno indica filas y dos indica columnas. El tercer argumento debe contener la función que se aplicará; en nuestro caso, la función `mean()` en la primera fila y la función `sd()` en la segunda fila. Los resultados se muestran a continuación:

```
> mean_data
      mpg  cylinders displacement  horsepower    weight acceleration
23.445918  5.471939 194.411990 104.469388 2977.584184 15.541327
> sd_data
      mpg  cylinders displacement  horsepower    weight acceleration
7.805007 1.705783 104.644004 38.491160 849.402560 2.758864
```


Para normalizar los datos, usamos la función `scale()`, que es una función genérica cuyo método predeterminado centra y/o escala las columnas de una matriz numérica:

```
> data_scaled <- as.data.frame(scale(data[,1:6],center = mean_data, scale = sd_data))
```

Echemos un vistazo a los datos transformados por normalización:

```
> head(data_scaled, n=20)
```

Los resultados son los siguientes:

	mpg	cylinders	displacement	horsepower	weight	acceleration
1	-0.69774672	1.4820530	1.07591459	0.6632851	0.6197483	-1.28361760
2	-1.08211534	1.4820530	1.48683159	1.5725848	0.8422577	-1.46485160
3	-0.69774672	1.4820530	1.18103289	1.1828849	0.5396921	-1.64608561
4	-0.95399247	1.4820530	1.04724596	1.1828849	0.5361602	-1.28361760
5	-0.82586959	1.4820530	1.02813354	0.9230850	0.5549969	-1.82731962
6	-1.08211534	1.4820530	2.24177212	2.4299245	1.6051468	-2.00855363
7	-1.21023822	1.4820530	2.48067735	3.0014843	1.6204517	-2.37102164
8	-1.21023822	1.4820530	2.34689042	2.8715843	1.5710052	-2.55225565
9	-1.21023822	1.4820530	2.49023356	3.1313843	1.7040399	-2.00855363
10	-1.08211534	1.4820530	1.86907996	2.2220846	1.0270935	-2.55225565
11	-1.08211534	1.4820530	1.80218649	1.7024847	0.6892089	-2.00855363
12	-1.21023822	1.4820530	1.39126949	1.4426848	0.7433646	-2.73348966
13	-1.08211534	1.4820530	1.96464205	1.1828849	0.9223139	-2.18978763
14	-1.21023822	1.4820530	2.49023356	3.1313843	0.1276377	-2.00855363
15	0.07099053	-0.8629108	-0.77799001	-0.2460146	-0.7129531	-0.19621355
16	-0.18525522	0.3095711	0.03428778	-0.2460146	-0.1702187	-0.01497955
17	-0.69774672	0.3095711	0.04384399	-0.1940546	-0.2396793	-0.01497955
18	-0.31337809	0.3095711	0.05340019	-0.5058145	-0.4598340	0.16625446
19	0.45535916	-0.8629108	-0.93088936	-0.4278746	-0.9978592	-0.37744756
20	0.32723628	-0.8629108	-0.93088936	-1.5190342	-1.3451622	1.79736053

Dividamos ahora los datos para el entrenamiento y la prueba:

```
> index = sample(1:nrow(data),round(0.70*nrow(data)))
> train_data <- as.data.frame(data_scaled[index,])
> test_data <- as.data.frame(data_scaled[-index,])
```

En la primera línea del código recién sugerido, el conjunto de datos se divide en 70:30, con la intención de usar el 70 por ciento de los datos a nuestra disposición para entrenar la red y el 30 por ciento restante para probar la red. En la segunda y tercera líneas, los datos del marco de datos denominado `data` se subdividen en dos nuevos marcos de datos, denominados `train_data` y `test_data`. Ahora tenemos que construir la función que se enviará a la red:

```
> n = names(data_scaled)
> f = as.formula(paste("mpg ~", paste(n[!n %in% "mpg"], collapse = " + ")))
```

En la primera línea, recuperamos todos los nombres de las variables en el dataframe `data_scaled`, usando la función `names()`. En la segunda línea, construimos la fórmula que usaremos para entrenar la red. ¿Qué representa esta fórmula?

Los modelos ajustados por la función `neuralnet()` se especifican en una forma simbólica compacta. El operador `~` es básico en la formación de tales modelos. Una expresión de la forma `y ~ modelo` se interpreta como una especificación de que la respuesta `y` es modelada por un predictor especificado simbólicamente por `modelo`. Dicho modelo consta de una serie de términos separados por operadores `+`. Los términos en sí consisten en nombres de variables y factores separados por: operadores. Dicho término se interpreta como la interacción de todas las variables y factores que aparecen en el término. Veamos la fórmula que establecimos:

```
> f
mpg ~ cylinders + displacement + horsepower + weight + acceleration
```

Ahora podemos construir y entrenar la red.

En el tema, que corresponde al aprendizaje de las redes neuronales multicapa, dijimos que **para elegir la cantidad óptima de neuronas, necesitamos saber que:**

- Una **pequeña cantidad de neuronas conducirá a un alto error para tu sistema**, ya que los factores predictivos pueden ser demasiado complejos para que los capture una pequeña cantidad de neuronas.
- Una **gran cantidad de neuronas sobre ajustará tus datos** de entrenamiento y **no se generalizará bien**
- **El número de neuronas en cada capa oculta debe estar en algún lugar entre el tamaño de la capa de entrada y la de salida**, potencialmente **la media**
- **La cantidad de neuronas en cada capa oculta no debe exceder el doble de la cantidad de neuronas de entrada**, ya que probablemente esté muy sobre ajustado en este punto

En este caso, tenemos cinco variables de entrada (`cylinders`, `displacement`, `horsepower`, `weight` y `acceleration`) y una variable de salida (`mpg`). Elegimos colocar tres neuronas en la capa oculta.

```
> net = neuralnet(f,data=train_data,hidden=3,linear.output=TRUE)
```

El argumento oculto acepta un vector con el número de neuronas para cada capa oculta, mientras que el argumento `linear.output` se usa para especificar si queremos hacer **regresión** (`linear.output = TRUE`) **o clasificación** (`linear.output = FALSE`).

El algoritmo utilizado en `neuralnet()`, de forma predeterminada, se basa en la retropropagación resiliente sin retroceso de peso y, además, modifica una tasa de aprendizaje, ya sea la tasa de aprendizaje asociada con el gradiente absoluto más pequeño (sag) o la tasa de aprendizaje más pequeña (slr) en sí. La función `neuralnet()` devuelve un objeto de clase `nn`. Un objeto de clase `nn` es una lista que contiene como máximo los componentes que se muestran en la siguiente tabla:

Componentes	Descripción
<code>call</code>	La llamada coincidente.
<code>response</code>	Extraída del argumento de datos.
<code>covariate</code>	Las variables extraídas del argumento de datos.
<code>model.list</code>	Una lista que contiene las covariables y las variables de respuesta (<code>response</code>) extraídas del argumento de la formula.
<code>err.fct</code>	La función de error.
<code>act.fct</code>	La función de activación.
<code>data</code>	El argumento de datos.
<code>net.result</code>	Una lista que contiene el resultado general de la red neuronal para cada repetición.
<code>weights</code>	Una lista que contiene los pesos ajustados de la red neuronal para cada repetición.
<code>generalized.weights</code>	Una lista que contiene los pesos generalizados de la red neuronal para cada repetición.
<code>result.matrix</code>	Una matriz que contiene el umbral alcanzado, los pasos necesarios, el error, AIC y BIC (si se calculan) y los pesos de cada repetición. Cada columna representa una repetición.
<code>startweights</code>	Una lista que contiene los pesos iniciales de la red neuronal para cada repetición.

Para producir resúmenes de resultados del modelo, usamos la función `summary()`:

```
> summary(net)
```

```

      Length Class      Mode
call           5  -none-   call
response       274  -none-  numeric
covariate     1370  -none-  numeric
model.list      2  -none-   list
err.fct         1  -none-  function
act.fct         1  -none-  function
linear.output   1  -none-  logical
data            6 data.frame list
exclude         0  -none-   NULL
net.result      1  -none-   list
weights         1  -none-   list
generalized.weights 1  -none-   list
startweights    1  -none-   list
result.matrix   25  -none-  numeric

```

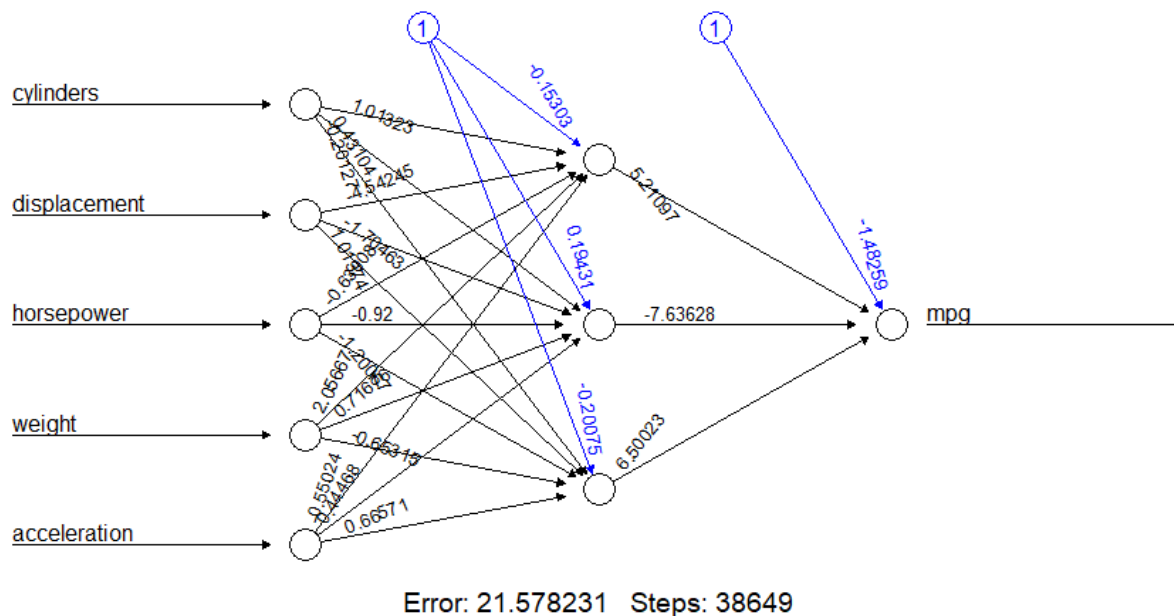
Para cada componente del modelo de red neuronal se muestran tres características:

- **Length**: Esta es la longitud del componente, es decir, cuántos elementos de este tipo están contenidos en él
- **Class**: Contiene una indicación específica sobre la clase de componente.
- **Mode**: Este es el tipo de componente (numérico, lista, función, lógico, etc.)

Para trazar la representación gráfica del modelo con los pesos en cada conexión, podemos usar la función `plot()`. La función `plot()` es una función genérica para la representación de objetos en R. Función genérica significa que es adecuada para diferentes tipos de objetos, desde variables hasta tablas y salidas de funciones complejas, produciendo diferentes resultados. Aplicado a una variable nominal, producirá un gráfico de barras. Aplicado a una variable cardinal, producirá un diagrama de dispersión. Aplicado a la misma variable, pero tabulada, es decir, a su distribución de frecuencias, producirá un histograma. Finalmente, aplicado a dos variables, una nominal y una cardinal, producirá un diagrama de caja.

```
> plot(net)
```

El plot de la red neuronal se muestra en la siguiente figura:



En la figura anterior, las líneas negras (estas líneas comienzan desde los nodos de entrada) muestran las conexiones entre cada capa y los pesos en cada conexión, mientras que las líneas azules (estas líneas comienzan desde los nodos de sesgo que se distinguen por el número 1) muestran el término sesgo agregado en cada paso. El sesgo se puede considerar como la intersección de un modelo lineal.

Aunque con el tiempo hemos entendido mucho sobre la mecánica que es la base de las redes neuronales, en muchos aspectos, el modelo que hemos construido y entrenado sigue siendo una caja negra. El ajuste, los pesos y el modelo no son lo suficientemente claros. Podemos estar satisfechos de que el algoritmo de entrenamiento sea convergente y luego el modelo esté listo para ser utilizado.

Podemos imprimir igualmente, los pesos y sesgos:

```
> net$result.matrix
```

```

                                [,1]
error                        2.157823e+01
reached.threshold           9.716452e-03
steps                       3.864900e+04
Intercept.to.l1ayhid1      -1.530296e-01
cylinders.to.l1ayhid1       1.013231e+00
displacement.to.l1ayhid1  -4.542445e+00
horsepower.to.l1ayhid1     -6.300786e-01
weight.to.l1ayhid1         2.056666e+00
acceleration.to.l1ayhid1  -5.502395e-01
Intercept.to.l1ayhid2       1.943091e-01
cylinders.to.l1ayhid2       4.310374e-01
displacement.to.l1ayhid2  -1.704629e+00
horsepower.to.l1ayhid2     -9.200019e-01
weight.to.l1ayhid2         7.167602e-01
acceleration.to.l1ayhid2   4.446811e-01
Intercept.to.l1ayhid3      -2.007484e-01
cylinders.to.l1ayhid3      -2.012653e-01
displacement.to.l1ayhid3   1.013737e+00
horsepower.to.l1ayhid3    -1.200674e+00
weight.to.l1ayhid3        -6.531474e-01
acceleration.to.l1ayhid3   6.657115e-01
Intercept.to.mpg           -1.482592e+00
l1ayhid1.to.mpg             5.210969e+00
l1ayhid2.to.mpg            -7.636277e+00
l1ayhid3.to.mpg            6.500225e+00

```

Como puede verse, estos son los mismos valores que podemos leer en el plot de la red. Por ejemplo, `cylinders.to.l1ayhid1 = 1.013231e+00` es el peso para la conexión entre los cilindros de entrada y el primer nodo de la capa oculta.

Ahora podemos usar la red para hacer predicciones. Para esto, habíamos reservado el 30 por ciento de los datos en el marco de datos `test_data`. Es hora de usarlo.

```
> predict_net_test <- compute(net,test_data[,2:6])
```

En nuestro caso, aplicamos la función al conjunto de datos `test_data`, usando solo las columnas de 2 a 6, que representan las variables de entrada de la red. Para evaluar el rendimiento de la red, podemos usar el error cuadrático medio (MSE) como una medida de qué tan lejos están nuestras predicciones de los datos reales.

```
> MSE.net <- sum((test_data$mpg - predict_net_test$net.result)^2)/nrow(test_data)
```

Aquí `test_data$mpg` son los datos reales y `predict_net_test$net.result` son los datos predichos para el objetivo del análisis. A continuación, se muestra el resultado:

```
> MSE.net
[1] 0.3217624
```

Parece un buen resultado, pero ¿con qué lo **comparamos**? Para tener una idea de la precisión de la predicción de la red, podemos construir un **modelo de regresión lineal**:

```
> Lm_Mod <- lm(mpg~., data=train_data)
> summary(Lm_Mod)
```

Construimos un modelo de regresión lineal usando la función `lm`. Esta función se utiliza para ajustar modelos lineales. Se puede utilizar para realizar regresiones, análisis de varianza de un solo estrato y análisis de covarianza. Para producir un resumen de los resultados del ajuste del modelo obtenidos, hemos utilizado la función `summary()`, que devuelve los siguientes resultados:

```
> summary(Lm_Mod)

Call:
lm(formula = mpg ~ ., data = train_data)

Residuals:
    Min       1Q   Median       3Q      Max
-1.47090 -0.33552 -0.04582  0.28098  1.78191

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.002720   0.031603  -0.086  0.931473
cylinders     -0.071339   0.102099  -0.699  0.485328
displacement -0.009649   0.136132  -0.071  0.943545
horsepower   -0.339331   0.097863  -3.467  0.000612 ***
weight       -0.479782   0.097648  -4.913  1.56e-06 ***
acceleration -0.069349   0.052648  -1.317  0.188886
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.5212 on 268 degrees of freedom
Multiple R-squared:  0.7155,    Adjusted R-squared:  0.7102
F-statistic: 134.8 on 5 and 268 DF,  p-value: < 2.2e-16
```

Ahora hacemos la predicción con el modelo de regresión lineal utilizando los datos contenidos en el marco de datos `test_data`:

```
> predict_lm <- predict(Lm_Mod, test_data)
```

Finalmente, calculamos el MSE para el modelo de regresión:

```
> MSE.lm <- sum((predict_lm - test_data$mpg)^2)/nrow(test_data)
```

A continuación, se muestra el resultado:

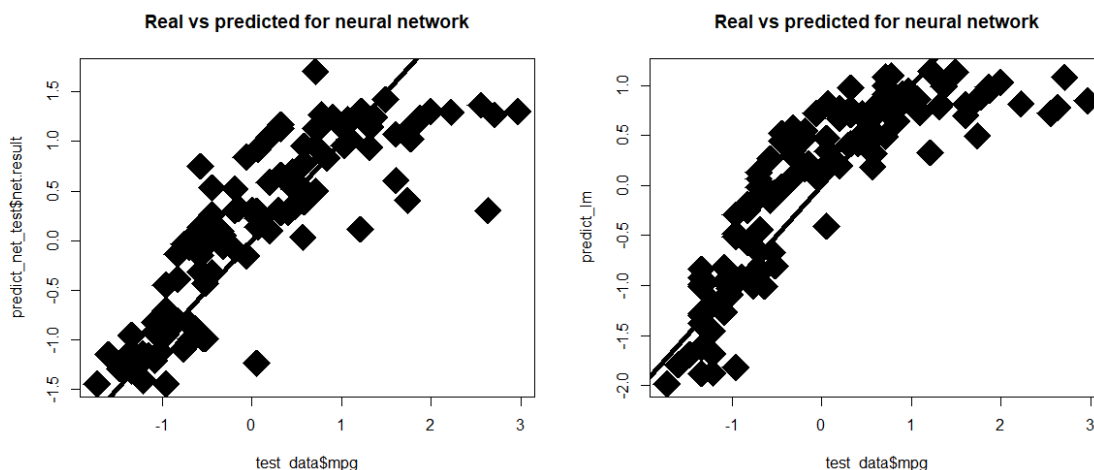
```
> MSE.lm
[1] 0.358526
```

De la comparación entre los dos modelos (modelo de red neuronal versus modelo de regresión lineal), una vez más, la red neuronal gana (0.32 versus 0.35).

Ahora realizamos una comparación visual dibujando en un gráfico el valor real frente al valor predicho, primero para la red neuronal y luego para el modelo de regresión lineal:

```
> par(mfrow=c(1,2))
> plot(test_data$mpg,predict_net_test$net.result,col='black',main='Real vs predicted for
neural network',pch=18,cex=4)
> abline(0,1,lwd=5)
> plot(test_data$mpg,predict_lm,col='black',main='Real vs predicted for neural
network',pch=18,cex=4)
> abline(0,1,lwd=5)
```

La comparación entre el rendimiento del modelo de red neuronal (a la izquierda) y el modelo de regresión lineal (a la derecha) en el conjunto de prueba se representa en la siguiente figura:



Como podemos ver, las predicciones de la red neuronal están más concentradas alrededor de la línea que las del modelo de regresión lineal, incluso si no nota una gran diferencia.